

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	M.Manasa	Reg. No.:	192425189
Section / Batch:	2024 AIML	Date:	26/8/2026

Code of Conduct

I M.MANASA(192425189) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: m.manasa

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A conversational AI system must identify grammatical relationships in user queries containing long-distance dependencies. The developers are comparing constituency parsing and dependency parsing.

Analyze how the two representations differ in capturing hierarchical and word-to-word relationships, and justify which approach is more suitable for extracting meaningful relationships from conversational text.

2. An NLP system processes partially typed search queries such as “best hotels near...” and “book a flight from Chennai to...”. The system currently uses conventional top-down parsing, while the developers are considering Earley parsing.

Analyze how the two approaches behave when input is incomplete or ambiguous, and justify which parsing strategy is more appropriate for interactive NLP applications.

3. A grammar-checking system must distinguish between multiple interpretations of structurally ambiguous sentences. The developers are comparing CFG, PCFG, and neural constituency parsers.

Analyze how each approach represents and resolves syntactic ambiguity, and justify which approach would provide the best balance between interpretability and practical accuracy.

4. A multilingual NLP system must process sentences in which grammatical information depends on number, gender, tense, and person. The developers are considering feature structures and traditional context-free grammar rules.

Analyze how feature structures extend basic grammar representations and justify their usefulness for enforcing grammatical constraints in multilingual systems.

5. A voice assistant must interpret commands containing verbs with different argument requirements, such as “give the book to John,” “sleep,” and “put the file on the desk.” The developers are considering subcategorization frames.

Analyze how subcategorization information helps determine valid sentence structures and justify its importance in semantic and syntactic analysis.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1	4	3	4	4	3	18	92
2	4	3	3	4	3	17	
3	4	4	3	3	4	18	
4	4	4	4	4	3	19	
5	4	4	4	4	4	20	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1: Constituency Parsing vs Dependency Parsing

1. Scope of the Comparison

The comparison covers two fundamentally different syntactic representations: constituency (phrase-structure) parsing, which groups words into nested hierarchical phrases (NP, VP, PP, etc.) governed by context-free grammar rules, and dependency parsing, which represents sentence structure as a set of directed binary grammatical relations (head → dependent) between individual words, with no intermediate phrasal nodes.

2. Detailed Comparison

Aspect	Constituency Parsing	Dependency Parsing
Representation	Hierarchical tree of nested phrases (NP, VP, PP) built from CFG rules	Flat graph of word-to-word links labelled with grammatical relations (nsubj, dobj, etc.)
Unit of analysis	Phrases/constituents	Individual words (head–dependent pairs)
Handling long-distance dependencies	Relation between distant words is implicit; must be recovered by traversing multiple phrase nodes	Relation is explicit and direct — a single edge links the two words regardless of distance
Robustness to free/flexible word order	Sensitive to word order since rules define fixed phrase templates	More robust — relations are defined by grammatical function, not position
Computational cost for real-time use	Higher — requires building and searching full phrase-structure trees	Lower — direct arc-factored parsing (e.g., transition-based parsers) runs in near-linear time
Output usefulness for relation/intent extraction	Requires post-processing to extract head-dependent relations from the tree	Directly yields subject–verb–object triples usable for intent/slot extraction
Typical use case	Grammar checking, phrase-level transformations, constituency-based generation	Information extraction, question answering, chatbots, machine translation

3. Evidence from the Scenario

- Consider the query: "The hotel that I booked last month near the airport was cancelled." The subject ("hotel") and the main verb ("was cancelled") are separated by an entire relative clause — a long-distance dependency.
- In a constituency tree, "hotel" and "was cancelled" are separated by several nested nodes (NP → relative clause → S), so recovering the subject–verb link requires traversing multiple levels of the tree.
- In dependency parsing, a single labelled arc (nsubjpass) directly connects "hotel" to "cancelled", regardless of how much material intervenes — this is the core strength dependency grammar was designed for.
- Empirically, transition-based dependency parsers (e.g., arc-eager, arc-standard) achieve near-linear time complexity, which matters for a conversational system that must parse in real time.

4. Critical Reasoning and Justification

Dependency parsing is the more suitable approach for this conversational AI system. Its word-to-word representation directly exposes grammatical relationships (subject, object, modifier) without requiring the tree traversal that constituency parsing demands, which is essential when queries contain long-distance dependencies such as relative clauses, coordination, or embedded questions. Dependency structures also degrade more gracefully on informal, elliptical, or partially ungrammatical conversational text — a common occurrence in chat and voice interfaces — because relations are defined functionally rather than by rigid phrase templates. Constituency parsing remains valuable where explicit phrasal boundaries are required (e.g., syntactic paraphrase generation), but for extracting meaningful relationships from conversational text, dependency parsing offers superior efficiency, robustness, and direct usability for downstream tasks such as intent detection and slot filling.

Question 2: Top-Down Parsing vs Earley Parsing

Scenario: An NLP system processes partially typed, incomplete search queries (e.g., "best hotels near..."). It currently uses top-down parsing and is considering Earley parsing.

1. Scope of the Comparison

The comparison examines two parsing strategies for context-free grammars: top-down (recursive-descent) parsing, which predicts derivations starting from the start symbol and matches them against the input, and Earley parsing, a chart-based dynamic-programming algorithm that maintains all partially and fully matched grammar states in parallel.

2. Detailed Comparison

Aspect	Top-Down Parsing	Earley Parsing
Parsing strategy	Goal-driven prediction from the start symbol, expanded recursively	Chart-based; simultaneously tracks predicted, scanned, and completed states for every input position
Handling of incomplete input	Expects a complete derivation; fails or must be specially engineered to report partial matches	Naturally maintains a chart of partial derivations, so it can report exactly what is grammatically expected next
Handling of ambiguity	Requires backtracking across alternative rule expansions — costly and can be exponential	Represents ambiguous parses compactly with shared sub-structures; polynomial time even for ambiguous grammars
Grammar restrictions	Cannot handle left-recursive rules without grammar rewriting	Handles any context-free grammar, including left recursion, without transformation
Time complexity	Exponential in the worst case (with backtracking)	$O(n^3)$ worst case, $O(n^2)$ for unambiguous grammars, linear for many practical grammars
Suitability for incremental/interactive use	Poor — designed for one-shot, complete-string parsing	Strong — the chart can be extended incrementally as each new token arrives, ideal for autocomplete

3. Evidence from the Scenario

- For "best hotels near...", the input stream is incomplete. A top-down parser, built to derive a complete sentence from the start symbol, cannot gracefully represent an unfinished PP ("near ____") — it either fails, waits for more input with no partial state retained, or requires ad hoc restart logic.
- An Earley parser's chart at each position already contains the set of grammar rules that are 'in progress' (dotted rules). At the point after "near", the chart shows an active state expecting an NP to complete the prepositional phrase — this state can directly drive next-word prediction/autocomplete.
- For "book a flight from Chennai to...", multiple continuations are grammatically valid (a city, an airport code, a date). Earley's parallel state tracking naturally represents this ambiguity without exponential blow-up, whereas top-down parsing would need to backtrack across each alternative.

4. Critical Reasoning and Justification

Earley parsing is clearly more appropriate for interactive NLP applications that must process incomplete or ambiguous input as the user types. Its chart-based design keeps track of every partially matched rule at every input position, which means the system always has a well-defined notion of "what has been recognised so far" and "what is expected next" — exactly what is needed to support autocomplete and query prediction. It also avoids the combinatorial backtracking cost that top-down parsing incurs on ambiguous grammars, and it removes the left-recursion restriction that would otherwise force awkward

grammar rewriting. Top-down parsing is simpler to implement and adequate for short, unambiguous, fully-formed inputs, but it is not well suited to the incremental, ambiguity-rich nature of live search-query parsing.

Question 3: CFG vs PCFG vs Neural Constituency Parsers

Scenario: A grammar-checking system must resolve structurally ambiguous sentences. The developers are comparing CFG, PCFG, and neural constituency parsers.

1. Scope of the Comparison

This comparison spans three generations of constituency parsing technology: plain Context-Free Grammars (rule-based, non-probabilistic), Probabilistic CFGs (rule-based with statistically learned rule probabilities), and neural constituency parsers (learned, context-sensitive span/attachment scoring using models such as RNNs or Transformers).

2. Detailed Comparison

Aspect	CFG	PCFG	Neural Constituency Parser
Ambiguity handling	Generates all valid parse trees with no way to prefer one	Ranks competing parses by probability derived from rule frequencies	Ranks/produces parses using learned contextual representations of the whole sentence
Basis of decision	Purely syntactic rules	Rule probabilities estimated from a treebank	Distributed word/context embeddings capturing lexical and semantic cues
Interpretability	Fully interpretable — every rule is explicit	Still interpretable — probabilities attach to explicit rules	Largely a black box; decisions are hard to trace to explicit rules
Disambiguation accuracy	None (no ranking mechanism)	Moderate — limited by treebank coverage and inability to use deep semantic context	High — state-of-the-art accuracy on benchmark treebanks
Data / resource needs	Only a grammar is needed	Requires an annotated treebank to estimate probabilities	Requires large annotated corpora and significant compute
Robustness to novel/out-of-domain sentences	High coverage but no preference among parses	Degrades when input differs from training-corpus statistics	Generally best generalisation, but can fail unpredictably (uninterpretable errors)

3. Evidence from the Scenario

- Classic PP-attachment ambiguity, "I saw the man with the telescope," has two valid parses under a plain CFG (telescope as instrument of "saw" vs. a modifier of "man") with no mechanism to choose between them.
- A PCFG trained on a treebank can assign a higher probability to the more frequent attachment pattern (e.g., VP-attachment for "with" + instrument nouns), giving a usable, explainable preference while still letting a grammar-checker point to the specific rule and probability behind its choice.
- A neural constituency parser (e.g., a self-attentive span parser) uses the surrounding context and lexical semantics of "telescope" and "saw" to resolve the same ambiguity with substantially higher accuracy on benchmarks such as the Penn Treebank, but cannot easily justify the decision to an end user or developer debugging a false positive.

4. Critical Reasoning and Justification

For a grammar-checking system, PCFG offers the best balance between interpretability and practical accuracy. Because probabilities are attached directly to explicit, human-readable grammar rules, a PCFG-based checker can both resolve ambiguity statistically and justify its output (e.g., "this attachment is preferred because it occurs far more frequently") — a property essential for a tool that must explain corrections to users and be debuggable by linguists/developers. Plain CFG is

inadequate on its own since it provides no disambiguation at all. Neural constituency parsers deliver the highest raw accuracy and should be preferred where accuracy alone is the priority (or used as a secondary re-ranker), but their lack of interpretability is a genuine drawback for a system whose core purpose is to explain grammatical decisions rather than merely produce them. A hybrid design — a PCFG backbone with a neural re-ranker used only to break close statistical ties — is a reasonable practical compromise.

Question 4: Feature Structures vs Traditional Context-Free Grammar

Scenario: A multilingual NLP system must enforce agreement based on number, gender, tense, and person. The developers are considering feature structures versus traditional CFG rules.

1. Scope of the Comparison

The comparison contrasts plain, category-only CFG rules (e.g., $S \rightarrow NP\ VP$) with feature structures — attribute–value matrices attached to grammar categories (e.g., NUM, PERS, GEN, TENSE) that are checked for compatibility via unification during parsing, as used in unification-based formalisms such as HPSG and LFG.

2. Detailed Comparison

Aspect	Traditional CFG	Feature Structures (Unification Grammar)
Representation of grammatical properties	Categories only (NP, VP); number/gender/tense not represented	Categories carry attribute–value pairs (e.g., [NUM: plural, GEN: fem, PERS: 3])
Enforcing agreement	Requires a separate rule for every valid combination (singular subject–singular verb, plural–plural, etc.)	A single rule enforces agreement generically via unification of shared feature values
Grammar size / scalability	Explodes combinatorially as more agreement dimensions (gender $\times$ number $\times$ person $\times$ tense) are added	Stays compact — new feature values extend the lexicon/features, not the rule count
Cross-linguistic extensibility	Each language needs largely separate rule sets to encode its agreement system	The same rule schema can be reused across languages by changing the feature inventory (e.g., adding GEN for French/Hindi)
Error detection	Ungrammatical agreement may still parse if a matching (but semantically wrong) rule exists	Unification fails explicitly when features clash, directly flagging agreement violations
Maintainability	Difficult — rule proliferation makes the grammar hard to update	Easier — constraints are localized to feature values, not duplicated rules

3. Evidence from the Scenario

- English: "the book is on the table" vs. "the books are on the table" — a plain CFG would need two separate $S \rightarrow NP\ VP$ rule instances (or lexical duplication) to keep singular/plural forms aligned; a feature structure instead unifies a single NUM feature between the subject NP and the verb, automatically rejecting *"*the books is"*.
- French: gender agreement between article, noun, and adjective (*"le grand livre"* vs. *"la grande maison"*) requires GEN and NUM to unify across three separate constituents — with plain CFG this would require duplicated rules for every gender/number combination and every constituent involved; feature unification handles it with one rule schema.
- A single multilingual parser can reuse largely the same phrase-structure backbone across English, French, and Hindi by simply changing which features (e.g., adding GEN and formality/PERS distinctions for Hindi) are unified, rather than rewriting the grammar per language.

4. Critical Reasoning and Justification

Feature structures are clearly the more suitable representation for a multilingual system with rich agreement requirements. They extend the basic phrase-structure grammar with attribute–value constraints that are checked by unification rather than encoded as an explosive number of duplicate rules, keeping the grammar compact, maintainable, and explicit about why a sentence is ungrammatical (a specific feature clash) rather than merely failing to parse. This is especially valuable across languages with different agreement systems (gender in French/Hindi, honorifics/person in many Asian languages), since the same rule schema can be reused with a different feature inventory. Traditional CFG remains simpler for languages/domains with minimal agreement, but does not scale to the combinatorial agreement space a genuinely multilingual system must support.

Question 5: Subcategorization Frames for Verb Argument Structure

Scenario: A voice assistant must interpret commands with verbs requiring different arguments ("give the book to John," "sleep," "put the file on the desk"). The developers are considering subcategorization frames.

1. Scope of the Comparison / Analysis

This question is analytical rather than a two-way comparison: it examines what subcategorization frames add over a bare phrase-structure grammar, and why they matter for both syntactic well-formedness checking and semantic role assignment in a command-driven voice assistant.

2. Detailed Analysis of Subcategorization Frames

Verb	Subcategorization Frame	Required Arguments	Effect on Parsing/Interpretation
sleep	Intransitive: V	No complement (only a subject)	"Sleep the file" is correctly rejected — no object slot is licensed
give	Ditransitive: V NP PP(to)/ NP NP	Theme + Recipient ("give the book to John")	Parser expects two arguments; a command missing the recipient ("give the book") is flagged as syntactically/semantically incomplete for full task execution
put	V NP PP(locative)	Theme + obligatory location ("put the file on the desk")	The locative PP is obligatory, not optional — "put the file" alone is ill-formed, correctly triggering a clarification request

3. Evidence from the Scenario

- Without subcategorization information, a generic grammar would allow "put the file" to parse as a complete VP (V + NP), even though "put" is semantically incomplete without a location — subcategorization frames mark the locative PP as obligatory, so the parser/semantic module can correctly flag the command as underspecified and prompt the user for the missing destination.
- For "give the book to John," the ditransitive frame directly tells the semantic analyzer which NP is the Theme ("the book") and which is the Recipient ("John"), enabling correct slot filling (source object, destination) without additional heuristics.
- For "sleep," the intransitive frame prevents the system from expecting or trying to fill an object slot, avoiding spurious parses such as treating a following word as a direct object.
- Subcategorization also resolves PP-attachment ambiguity: because "put" specifically subcategorizes for a locative PP, "on the desk" is attached to the verb phrase (as an argument) rather than being mis-analyzed as an optional NP modifier.

4. Critical Reasoning and Justification

Subcategorization frames are essential to this voice assistant at both the syntactic and semantic levels. Syntactically, they let the parser validate whether a command contains all the arguments a given verb requires, correctly distinguishing complete commands from underspecified ones (e.g., "put the file" vs. "put the file on the desk") rather than silently accepting an incomplete structure. Semantically, they provide a direct, principled mapping from syntactic arguments (NP, PP) to semantic roles (Agent, Theme, Recipient, Location) that the assistant's action-execution module needs to actually carry out the command — for instance, a file-move action requires both a source (Theme) and a destination (Location), and subcategorization frames guarantee both slots are identified and validated before execution. Without this information, the system would either over-generate ill-formed interpretations or under-specify the arguments needed for correct task execution, making subcategorization frames indispensable for reliable command understanding.